

AEGIS MOBILE

SwiftUI through a React lens

Part 01 / Sign-in architecture

A code-grounded guide for the first iOS slice

30 SEPTEMBER 2026

What exists today

0

Swift source files in apps/ios/

The Spring API already implements sign-in and /auth/me.
The iOS image at right is approved concept art, not a running app.



APPROVED VISUAL DIRECTION / CONCEPT ONLY

React ideas, SwiftUI names

Familiar idea

SwiftUI tool

Function component

struct View + body

useState

@State for local UI state

props

stored view properties

shared observable state

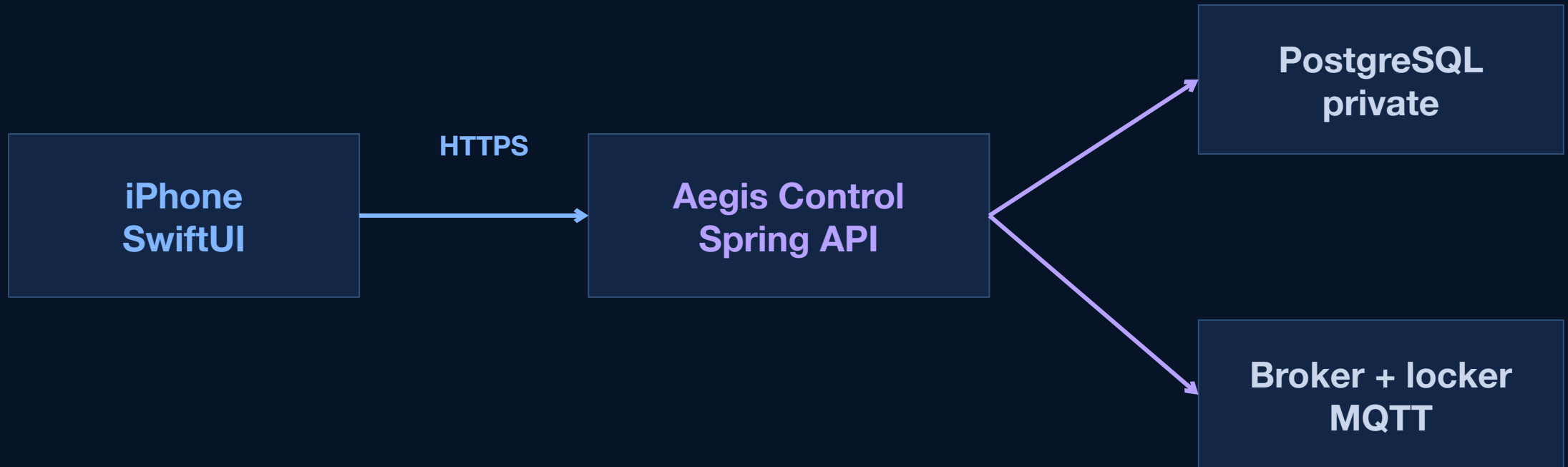
@Observable model (iOS 17+)

effect on appearance

.task for async work

The mapping helps you learn. SwiftUI owns rendering and state lifetimes differently from React.

The iOS trust boundary



The phone asks. The backend decides. The locker executes authorised commands.

The first HTTP contract

POST /api/v1/auth/login

```
{  
  "email": "technician@aegis.demo",  
  "password": "<entered by user>"  
}
```

200 response

```
{  
  "accessToken": "<opaque>",  
  "expiresIn": 3600,  
  "user": { "role": "TECHNICIAN" }  
}
```

Then GET /api/v1/auth/me with Authorization: Bearer <token>.

The app reads the profile from the server; it never derives roles by decoding the JWT.

Swift models mirror the wire format

```
// Illustrative Swift; not in the repository yet
struct LoginRequest: Encodable {
    let email: String
    let password: String
}

struct UserProfile: Decodable {
    let id: UUID
    let displayName: String
    let email: String
    let role: String
    let maximumAccessLevel: String
}
```

Why Codable?

It maps the documented JSON fields to Swift values. Keep the token as an opaque string and keep business decisions on the server.

Contract change? Update the server DTO, the REST document, the client model and tests together.

One API client owns networking

```
// Illustrative Swift; validate(...) is app-defined
func login(_ input: LoginRequest) async throws -> LoginResponse {
    let url = baseURL.appending(path: "api/v1/auth/login")
    var request = URLRequest(url: url)
    request.httpMethod = "POST"
    request.httpBody = try encoder.encode(input)
    request.setValue("application/json", forHTTPHeaderField: "Content-Type")

    let (data, response) = try await session.data(for: request)
    try validate(response, data)
    return try decoder.decode(LoginResponse.self, from: data)
}
```

Inject URLSession and the base URL. Test 200, 401, 429 and transport failure.

Session state across app launches

SIGN IN	POST /auth/login	Save accessToken in Keychain
REOPEN	Read Keychain token	GET /auth/me to refresh profile
401	Token invalid or expired	Delete local token; show sign-in
SIGN OUT	User chooses logout	Delete local token

No refresh token in P0. Losing the network does not cancel an operation already authorised by Spring.

Sign-in UI as explicit states

- | | | |
|-----------|----------------------|---|
| 01 | Idle | Form accepts email and password |
| 02 | Submitting | Disable duplicate action; show progress |
| 03 | Rejected | Show a stable error, retain entered email |
| 04 | Authenticated | Show technician navigation after the server profile |

Use native text controls, Dynamic Type and VoiceOver labels. Glass belongs around the flow, never over critical text.

The first iOS slice

Build

One SwiftUI login view, an injected API client, Codable DTOs and a small Keychain adapter.

Verify

A real iPhone signs in, relaunches into /auth/me, rejects a wrong-role account and recovers from 401 or network loss.

Next lesson

Catalogue + personal readiness. The app displays the backend decision; it never calculates READY locally.